CALIFORNIA STATE UNIVERSITY
LONG BEACH

CECS 341: Computer Architecture & Organization

Hossein Sayadi, PhD

Department of Computer Engineering & Computer Science
California State University, Long Beach

Fall 2025

Pipeline Hazards

Lecture Outline

- Introduction to pipeline hazards
- Resource (or Structural) hazards
- Data hazards
- Control Hazards

Pipeline Hazards

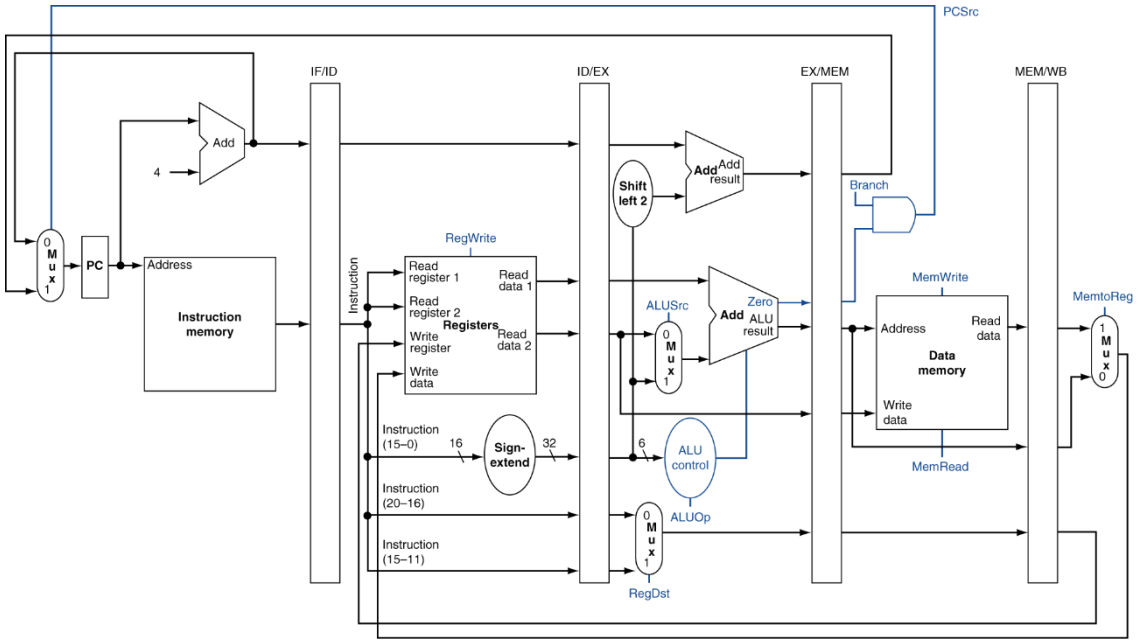
- There are three types of *pipeline hazards*:
  1. Resource (or Structural) Hazards
     - Due to contention for resources in the processor datapath.
  2. Data Hazards
     - Due to data dependency between instructions.
  3. Control (or Branch) Hazards
     - Due to branch instructions in program.
- Hazards reduce processor performance.

- Ideal pipeline (No hazards).
  - One instruction completed per clock cycle.
- Non-ideal pipeline (due to hazards).
  - Less than one instruction completed per clock cycle.

$IPC_{IDEAL} = 1$
$CPI_{IDEAL} = 1$

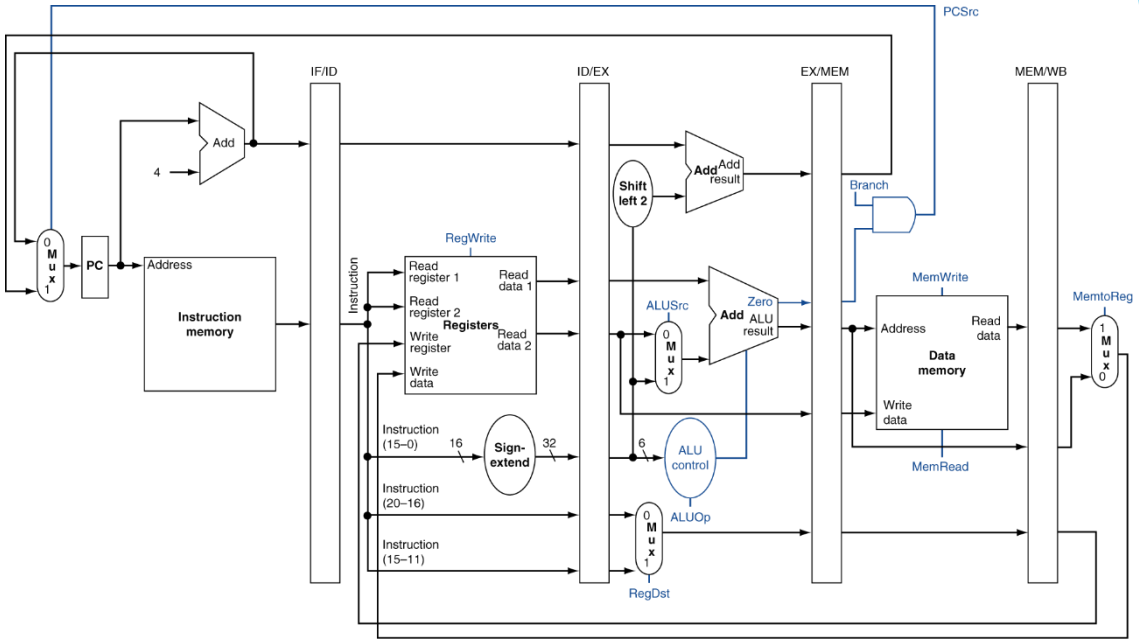
$CPI_{NON\text{-}IDEAL} > 1$

Pipeline Hazards



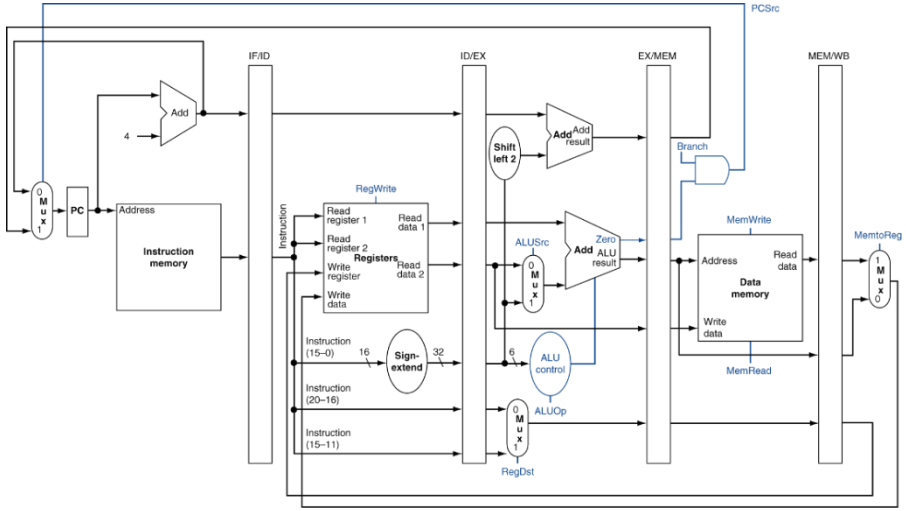
- All pipeline stages are synchronized to the same clock.
- Each stage processes an instruction during each clock cycle.
- At end of clock cycle, instruction passed to next pipeline stage.
  - Results of instruction processing written to pipeline register.
  - Contents of pipeline register used as input for next pipeline stage.

Pipeline Hazards



- Pipeline ***stalled*** when instruction cannot proceed to next stage.
- Processor performance reduced when pipeline stalled.
- A pipeline stall is implemented as a *NOP* (no operation).
  - Pipeline stage processing NOP does nothing for one clock cycle.
  - Corresponding pipeline register remains unchanged from previous cycle.

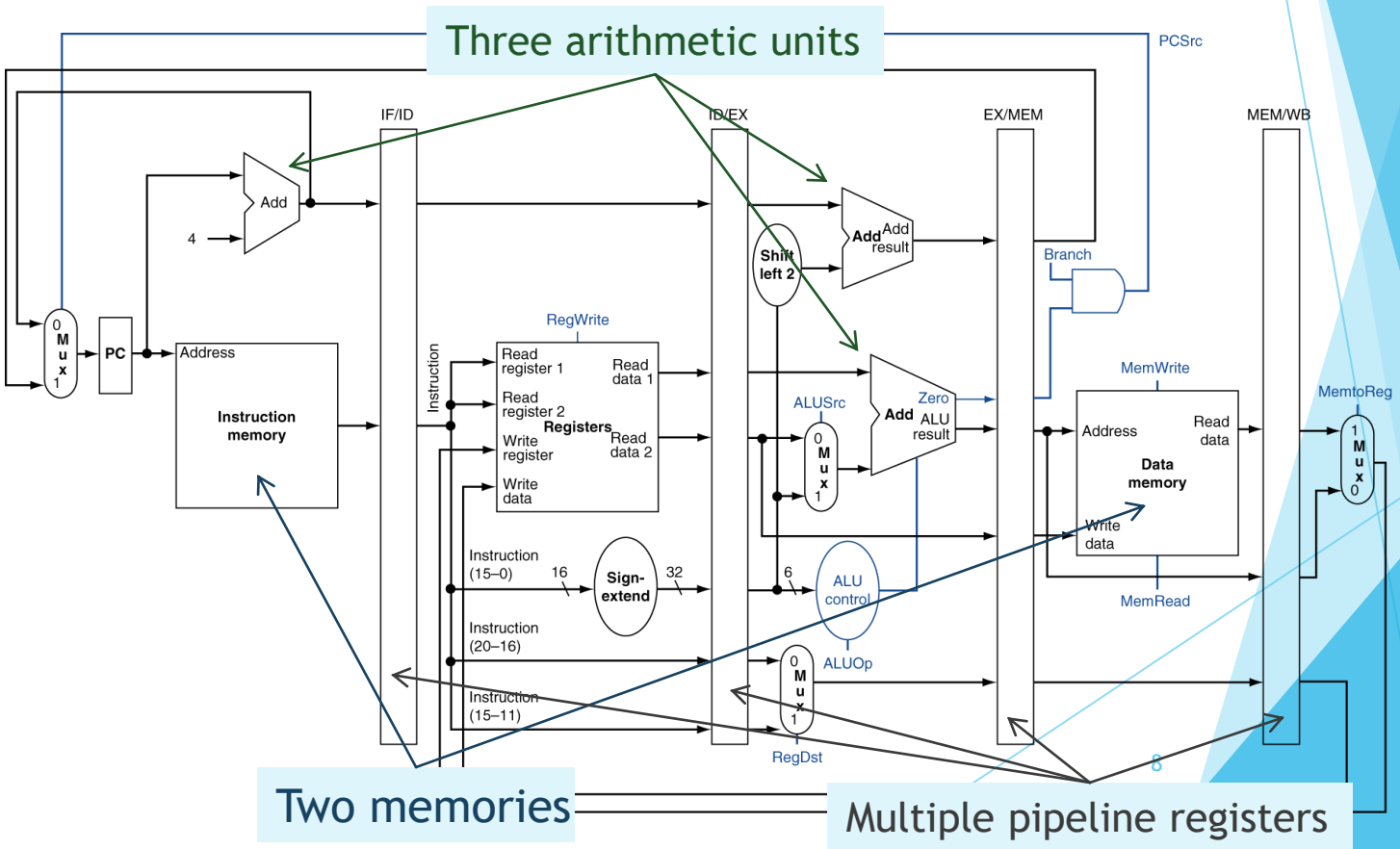
Resource Hazard



- Resource (Structural) Hazard: Conflict for the same processor resource.
- Occurs when two (or more) instructions that are already in pipeline require the same resource at the same time.
  - The result is that instruction must be executed in series rather than parallel for a portion of pipeline.
- Resource hazards
  - Are caused by a limitation in hardware resources.
  - Prevent concurrent operation of multiple pipeline stages.
  - Prevent concurrent execution of multiple instructions.

Resource Hazard

- Resource hazards can be eliminated by including additional hardware components in the processor datapath.



Data Hazard

- There are three types of data hazards:

1. Read after Write (RAW)

```
I1: add $s0, $s1, $s2
I2: add $s4, $s0, $s3
```

Instruction `I2` reads a register from the register file after instruction `I1` has written to the register.

2. Write after Read (WAR)

```
I1: add $s0, $s1, $s2
I2: add $s1, $s3, $s4
```

Instruction `I2` writes to a register in the register file after instruction `I1` has read the old value from the register.

3. Write after Write (WAW)

```
I1: add $s2, $s1, $s0
I2: add $s2, $s4, $s3
```

Instruction `I2` overwrites the value in a register previously written by instruction `I1`.

Data Hazard (RAW)

- Caused by a data dependency between two instructions.
  - An instruction produces a result that a subsequent instruction needs.

```
add $s0, $s1, $s2
add $s4, $s0, $s3
```

- Instructions need not be adjacent to one another.
  - May be separated by one or more unrelated instructions.

- Data dependencies do not cause a problem if instructions are executed one at a time.
  - That is, in a non-pipelined processor.
  - First instruction is completed before dependent instruction is fetched.

Data Hazard (RAW)

- However, data dependencies can cause problems in a pipelined processor.
  - First instruction and dependent instruction in pipeline at same time.
  - Instructions being executed concurrently.
  - Result (or data) needed by dependent instruction not yet available.
- An *arithmetic/logic* instruction can cause a data hazard.

```
add  $s0, $s1, $s2
addi $s4, $s0, 1
```

Result produced by *add* instruction needed by *addi* instruction.

- Similarly, a *load word* instruction can cause a data hazard.

```
lw   $s0, 12($t1)
add  $s2, $s0, $s1
```

Data read from memory by *load word* instruction needed by *add* instruction.

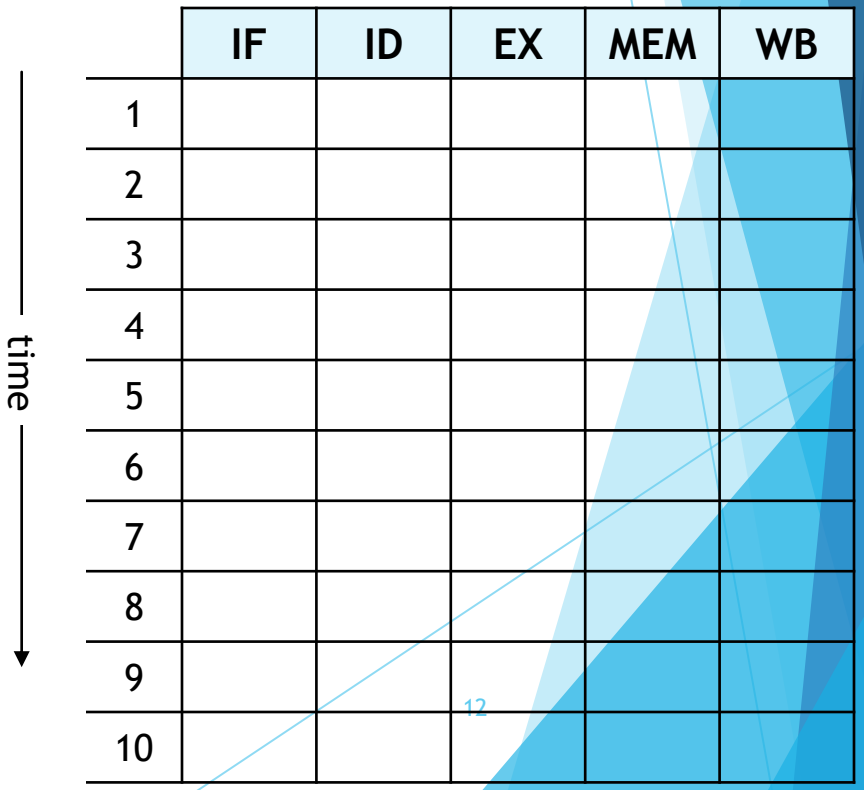
In the MIPS 5-stage pipeline, a data dependency between an instruction and either of the ***two*** subsequent instructions will result in a *Read after Write* (RAW) data hazard.

Data Hazard (RAW): Stalling

- RAW data hazards can be resolved by *stalling* the pipeline.
- Stall dependent instruction in ID stage of the pipeline until proper register value is available (i.e. ALU result).

```
I1: add $s0, $s1, $s2
I2: add $s4, $s0, $s3
I3: add $t0, $zero, $zero
I4: add $t1, $zero, $zero
I5: add $t2, $zero, $zero
```

- Pipeline stall
  - Insert NOP into pipeline ahead of stalled instruction.
  - Hold instructions in pipeline behind stalled instruction.



Data Hazard (RAW): Stalling

- RAW data hazards can be resolved by *stalling* the pipeline.
- Stall dependent instruction in ID stage of the pipeline until proper register value is available (i.e. ALU result).

```
I1: add $s0, $s1, $s2
I2: add $s4, $s0, $s3
I3: add $t0, $zero, $zero
I4: add $t1, $zero, $zero
I5: add $t2, $zero, $zero
```

Instruction `I1` is fetched from memory.

Clock Cycle #1

time

	IF	ID	EX	MEM	WB
1	I1				
2					
3					
4					
5					
6					
7					
8					
9					
10					

Data Hazard (RAW): Stalling

- RAW data hazards can be resolved by *stalling* the pipeline.
- Stall dependent instruction in ID stage of the pipeline until proper register value is available (i.e. ALU result).

```
I1: add $s0, $s1, $s2
I2: add $s4, $s0, $s3
I3: add $t0, $zero, $zero
I4: add $t1, $zero, $zero
I5: add $t2, $zero, $zero
```

Instruction `I2` is fetched from memory.
Instruction `I1` is decoded.

Clock Cycle #2

time

	IF	ID	EX	MEM	WB
1	I1				
2	I2	I1			
3					
4					
5					
6					
7					
8					
9					
10					

Data Hazard (RAW): Stalling

- RAW data hazards can be resolved by *stalling* the pipeline.
- Stall dependent instruction in ID stage of the pipeline until proper register value is available (i.e. ALU result).

```
I1: add $s0, $s1, $s2
I2: add $s4, $s0, $s3
I3: add $t0, $zero, $zero
I4: add $t1, $zero, $zero
I5: add $t2, $zero, $zero
```

Instruction `I3` is fetched from memory.
Instruction `I2` is decoded.
Instruction `I1` calculates sum.

Clock Cycle #3

time

	IF	ID	EX	MEM	WB
1	I1				
2	I2	I1			
3	I3	I2	I1		
4					
5					
6					
7					
8					
9					
10					

Data Hazard (RAW): Stalling

- RAW data hazards can be resolved by *stalling* the pipeline.
- Stall dependent instruction in ID stage of the pipeline until proper register value is available (i.e. ALU result).

```
I1: add $s0, $s1, $s2
I2: add $s4, $s0, $s3
I3: add $t0, $zero, $zero
I4: add $t1, $zero, $zero
I5: add $t2, $zero, $zero
```

Instruction `I2` is stalled in ID stage.
Until instruction `I1` writes sum ($s0) to register file (in WB stage).

Clock Cycle #4

time

	IF	ID	EX	MEM	WB
1	I1				
2	I2	I1			
3	I3	I2	I1		
4	I3	**I2**	NOP	I1	
5					
6					
7					
8					
9					
10					

Data Hazard (RAW): Stalling

- RAW data hazards can be resolved by *stalling* the pipeline.
- Stall dependent instruction in ID stage of the pipeline until proper register value is available (i.e. ALU result).

```
I1: add $s0, $s1, $s2
I2: add $s4, $s0, $s3
I3: add $t0, $zero, $zero
I4: add $t1, $zero, $zero
I5: add $t2, $zero, $zero
```

Instruction `I1` writes register during first half of clock cycle.
Instruction `I2` reads register during second half of clock cycle.

Clock Cycle #5

time

	IF	ID	EX	MEM	WB
1	I1				
2	I2	I1			
3	I3	I2	I1		
4	I3	I2	NOP	I1	
5	I3	**I2**	NOP	NOP	**I1**
6					
7					
8					
9					
10					

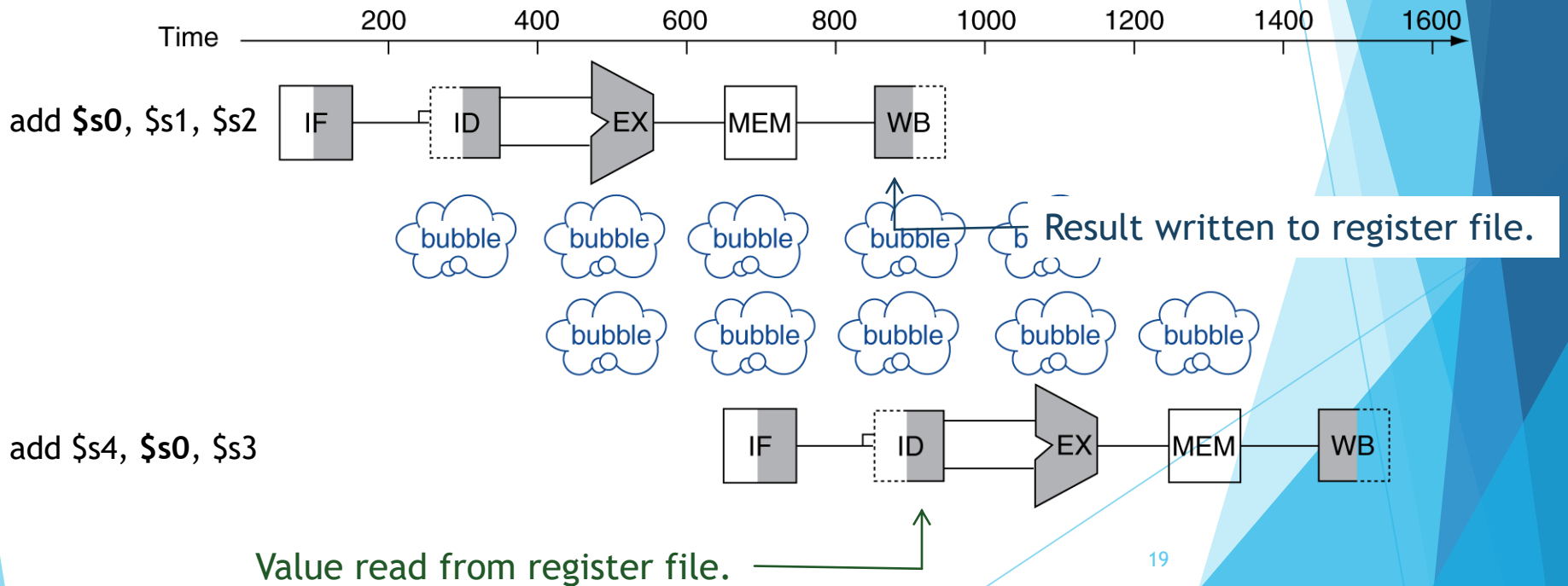
Data Hazard (RAW): Stalling

- RAW data hazards can be resolved by *stalling* the pipeline.
- Stall dependent instruction in ID stage of the pipeline until proper register value is available.
  - Prior instruction writes result to register file in WB stage.
- A *pipeline stall* is implemented as follows:
  - Insert "bubble" (or NOP) into the pipeline ahead of the stalled instruction.
  - Hold instructions in the pipeline behind the stalled instruction.

	1	2	3	4	5	6	7	8	9	10
add **$s0**, $s1, $s2	IF	ID	EX	MEM	**WB**					
NOP		IF	ID	NOP	NOP	NOP				
NOP			IF	ID	NOP	NOP	NOP			
add $s4, **$s0**, $s3				IF	**ID**	EX	MEM	WB		
add $t0, $0, $0					IF	ID	EX	MEM	WB	

Data Hazard (RAW): Stalling

- For the MIPS pipeline:
  - Data is written to a register during the first half of the clock cycle,
  - And read from a register during the second half of the clock cycle.

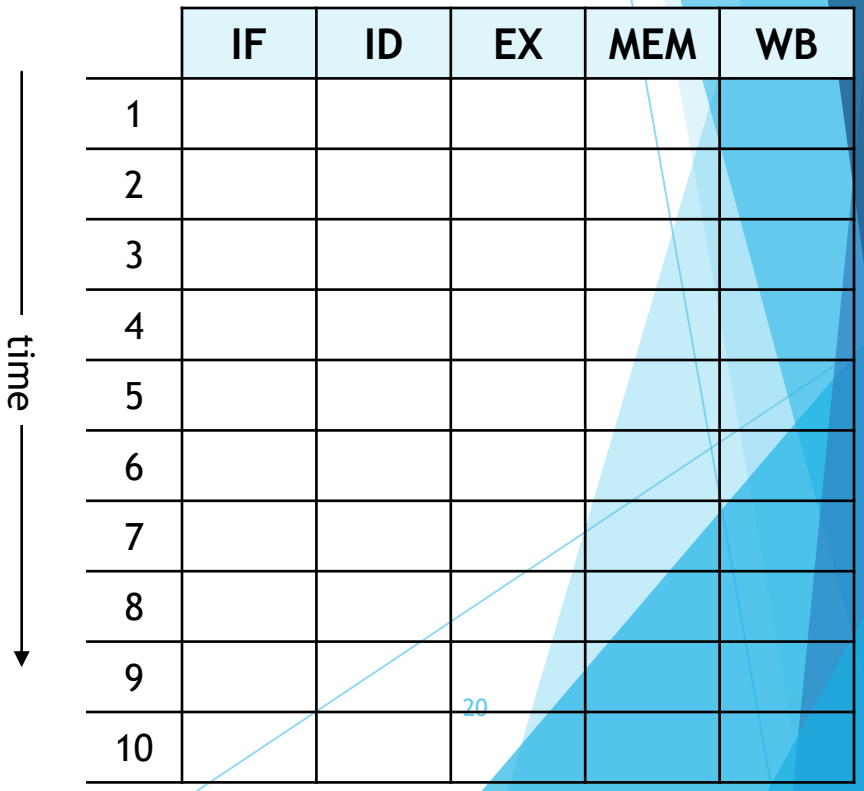


Data Hazard (RAW): Stalling

- RAW data hazards can be resolved by *stalling* the pipeline.
- Stall dependent instruction in ID stage of the pipeline until proper register value is available (i.e. data read from memory).

```
I1: lw  $s0, 12($t1)
I2: add $s4, $s0, $s3
I3: add $t0, $zero, $zero
I4: add $t1, $zero, $zero
I5: add $t2, $zero, $zero
```

- Pipeline stall
  - Insert NOP into pipeline ahead of stalled instruction.
  - Hold instructions in pipeline behind stalled instruction.



Data Hazard (RAW): Stalling

- RAW data hazards can be resolved by *stalling* the pipeline.
- Stall dependent instruction in ID stage of the pipeline until proper register value is available (i.e. data read from memory).

```
I1: lw  $s0, 12($t1)
I2: add $s4, $s0, $s3
I3: add $t0, $zero, $zero
I4: add $t1, $zero, $zero
I5: add $t2, $zero, $zero
```

Instruction `I3` is fetched from memory.
Instruction `I2` is decoded.
Instruction `I1` calculates mem address.

Clock Cycle #3

time

	IF	ID	EX	MEM	WB
1	I1				
2	I2	I1			
3	I3	I2	I1		
4					
5					
6					
7					
8					
9					
10					

Data Hazard (RAW): Stalling

- RAW data hazards can be resolved by *stalling* the pipeline.
- Stall dependent instruction in ID stage of the pipeline until proper register value is available (i.e. data read from memory).

```
I1: lw  $s0, 12($t1)
I2: add $s4, $s0, $s3
I3: add $t0, $zero, $zero
I4: add $t1, $zero, $zero
I5: add $t2, $zero, $zero
```

Instruction `I2` is stalled in ID stage.
Instruction `I1` reads data from memory, but does not write it to the register file until the next clock cycle (in WB stage).

Clock Cycle #4

time

	IF	ID	EX	MEM	WB
1	I1				
2	I2	I1			
3	I3	I2	I1		
4	I3	**I2**	NOP	I1	
5					
6					
7					
8					
9					
10					

Data Hazard (RAW): Stalling

- RAW data hazards can be resolved by *stalling* the pipeline.
- Stall dependent instruction in ID stage of the pipeline until proper register value is available (i.e. data read from memory).

```
I1: lw  $s0, 12($t1)
I2: add $s4, $s0, $s3
I3: add $t0, $zero, $zero
I4: add $t1, $zero, $zero
I5: add $t2, $zero, $zero
```

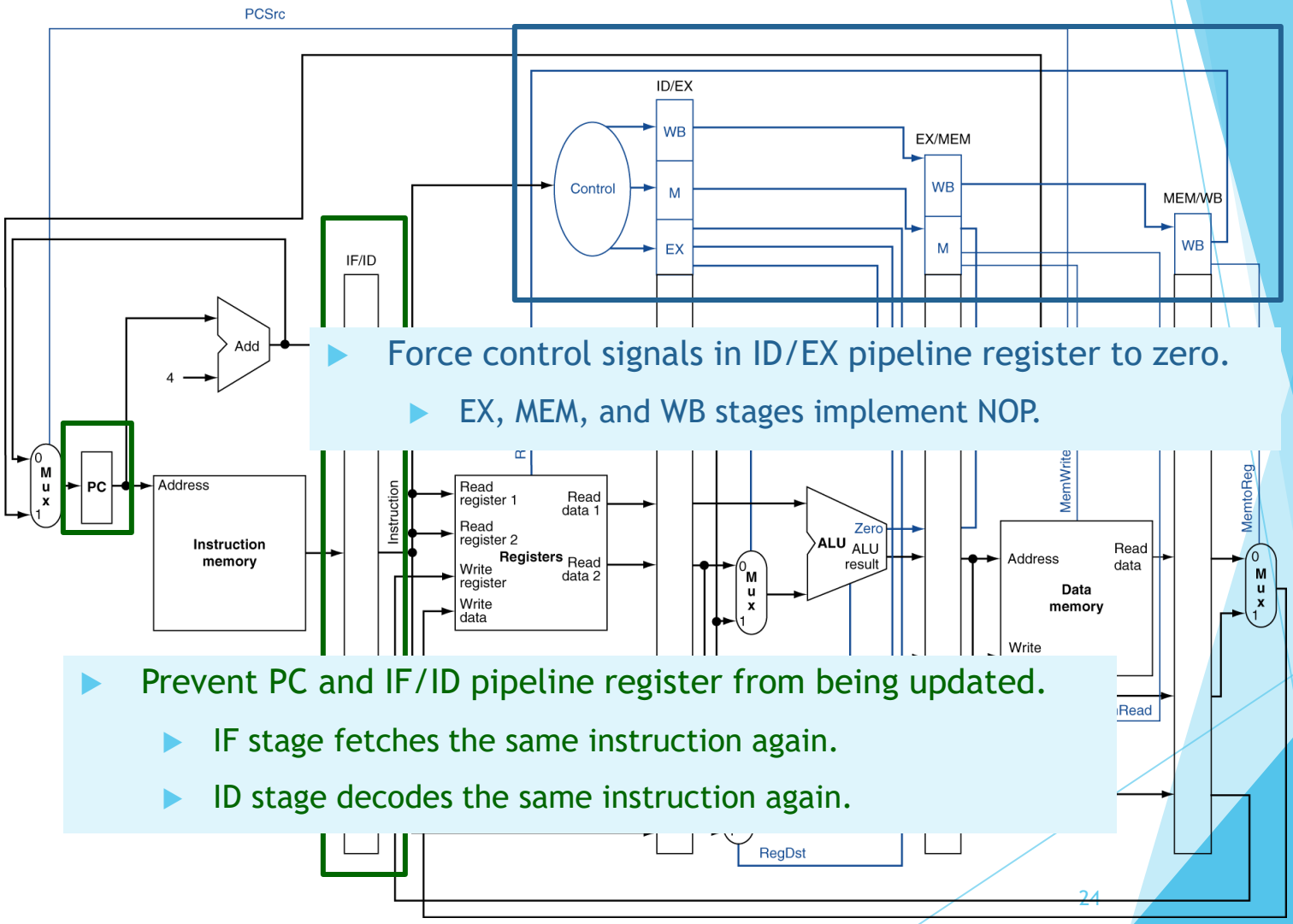
Instruction `I1` writes register during first half of clock cycle.
Instruction `I2` reads register during second half of clock cycle.

Clock Cycle #5

time

	IF	ID	EX	MEM	WB
1	I1				
2	I2	I1			
3	I3	I2	I1		
4	I3	I2	NOP	I1	
5	I3	**I2**	NOP	NOP	**I1**
6					
7					
8					
9					
10					

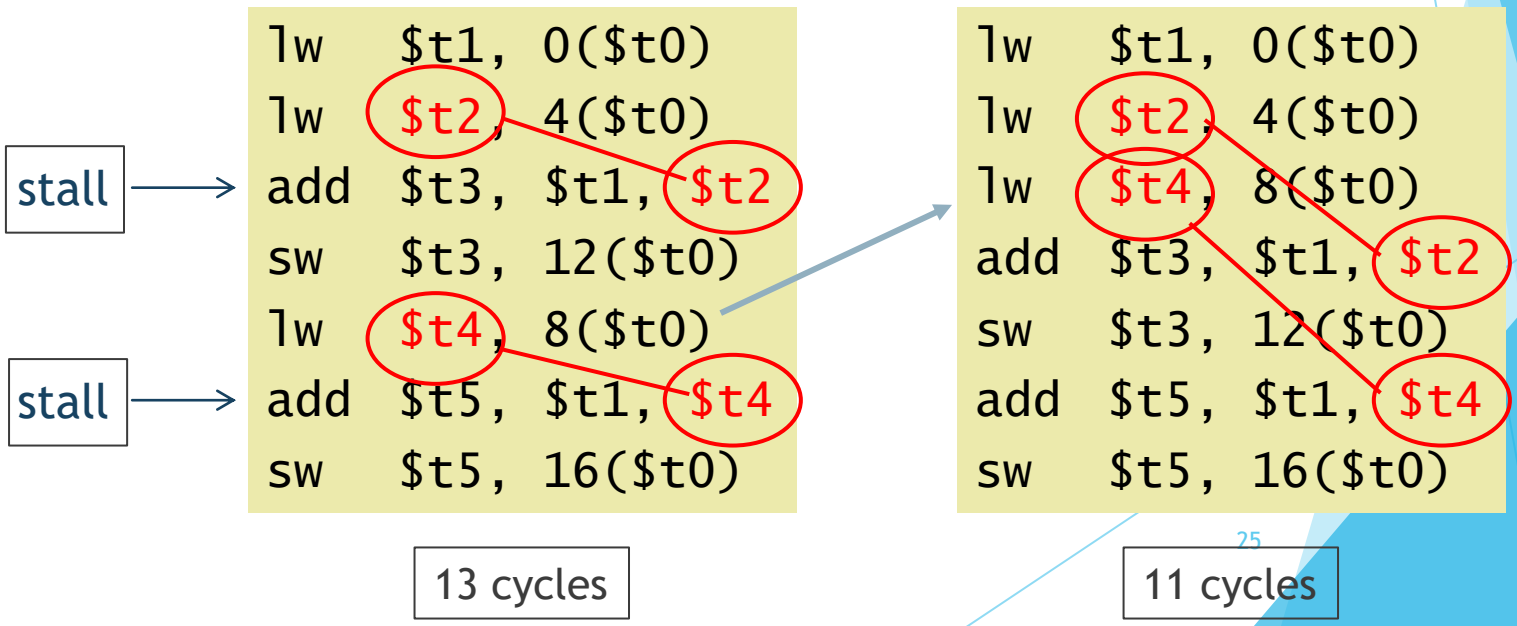
Hazards: Stalling the Pipeline



- Force control signals in ID/EX pipeline register to zero.
  - EX, MEM, and WB stages implement NOP.
- Prevent PC and IF/ID pipeline register from being updated.
  - IF stage fetches the same instruction again.
  - ID stage decodes the same instruction again.

Code Scheduling to Avoid Stalls

- Pipeline needs to be stalled for one clock cycle when result of *load word* instruction is used by next instruction.
- We can reorder code to avoid use of *load word* result in next instruction.
  - Eliminates the requirement for pipeline stall.
  - Results in a more efficient code execution process.

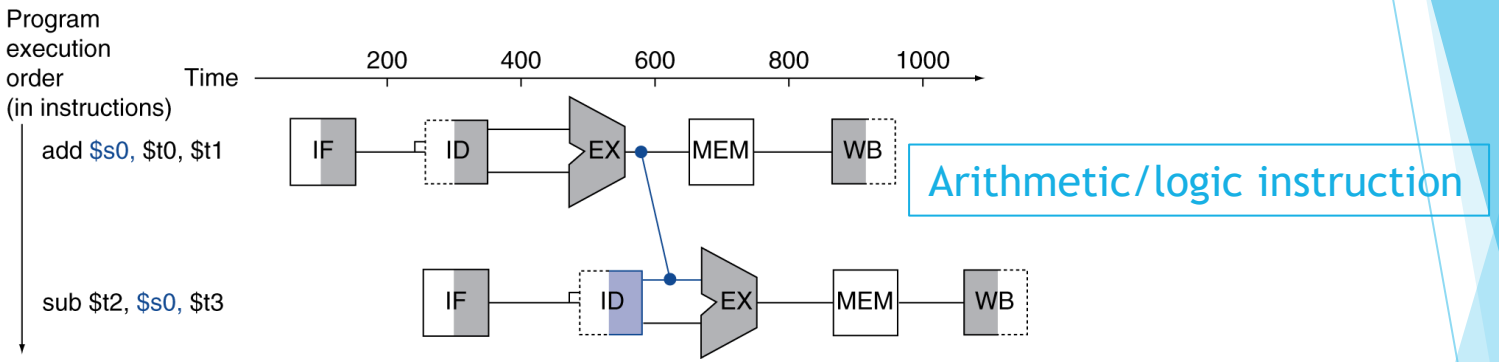


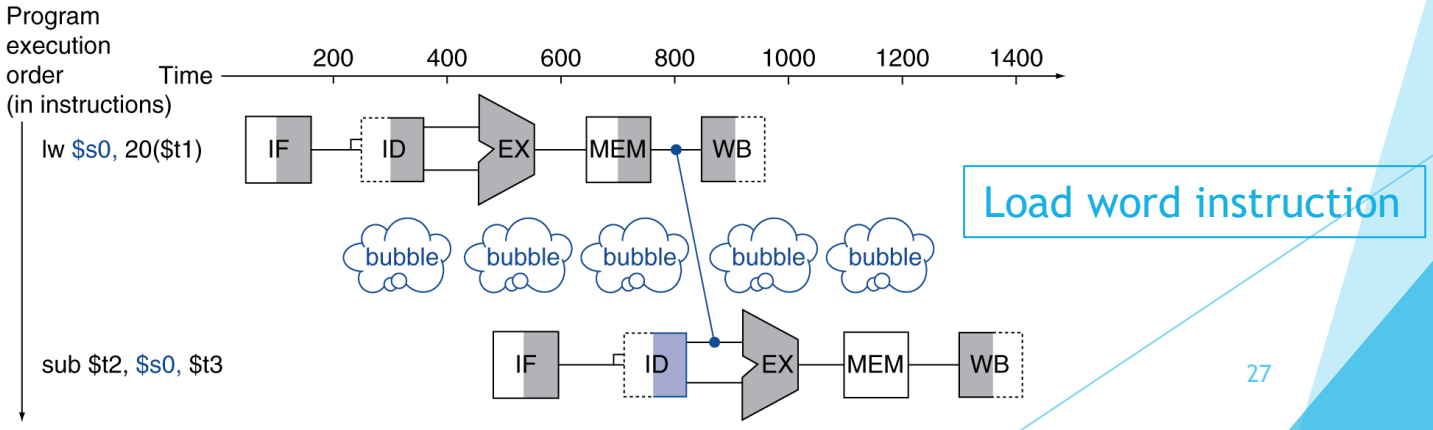
Data Hazard (RAW): Forwarding

- Objective of *forwarding* is to minimize the number of clock cycles that the pipeline must be stalled.
- Do ***not*** wait for data to be written back to the register file.
  - Destination register is written in the WB pipeline stage.
- Instead, data can be *forwarded* to the instruction that needs it.
  - ALU result is generated by instruction in the EX pipeline stage.
  - Data is read from memory in the MEM pipeline stage.
  - Result/data is needed by dependent instruction in the ID pipeline stage.
  - *Forward* the result/data to the instruction that needs it ***without*** first writing it back to the destination register in the register file.
- *Forwarding* requires additional connections in the datapath.

Data Hazard (RAW): Forwarding

- *Forwarding* illustrated using multiple-cycle pipeline diagrams.

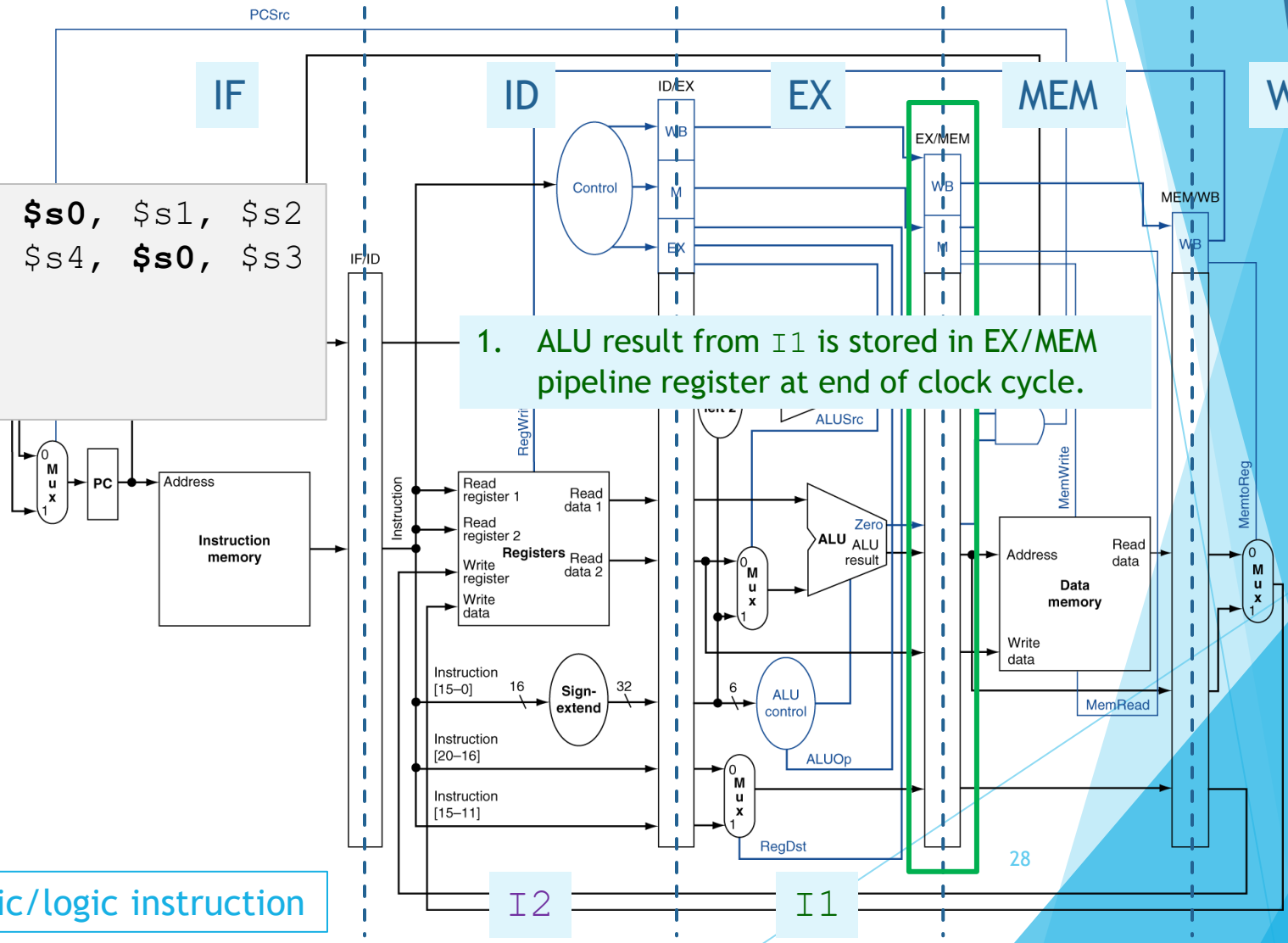




Forwarding: Arith/Logic Instruction

```
I1: add $s0, $s1, $s2
I2: add $s4, $s0, $s3
I3: …
I4: …
I5: …
```

1. ALU result from `I1` is stored in EX/MEM pipeline register at end of clock cycle.

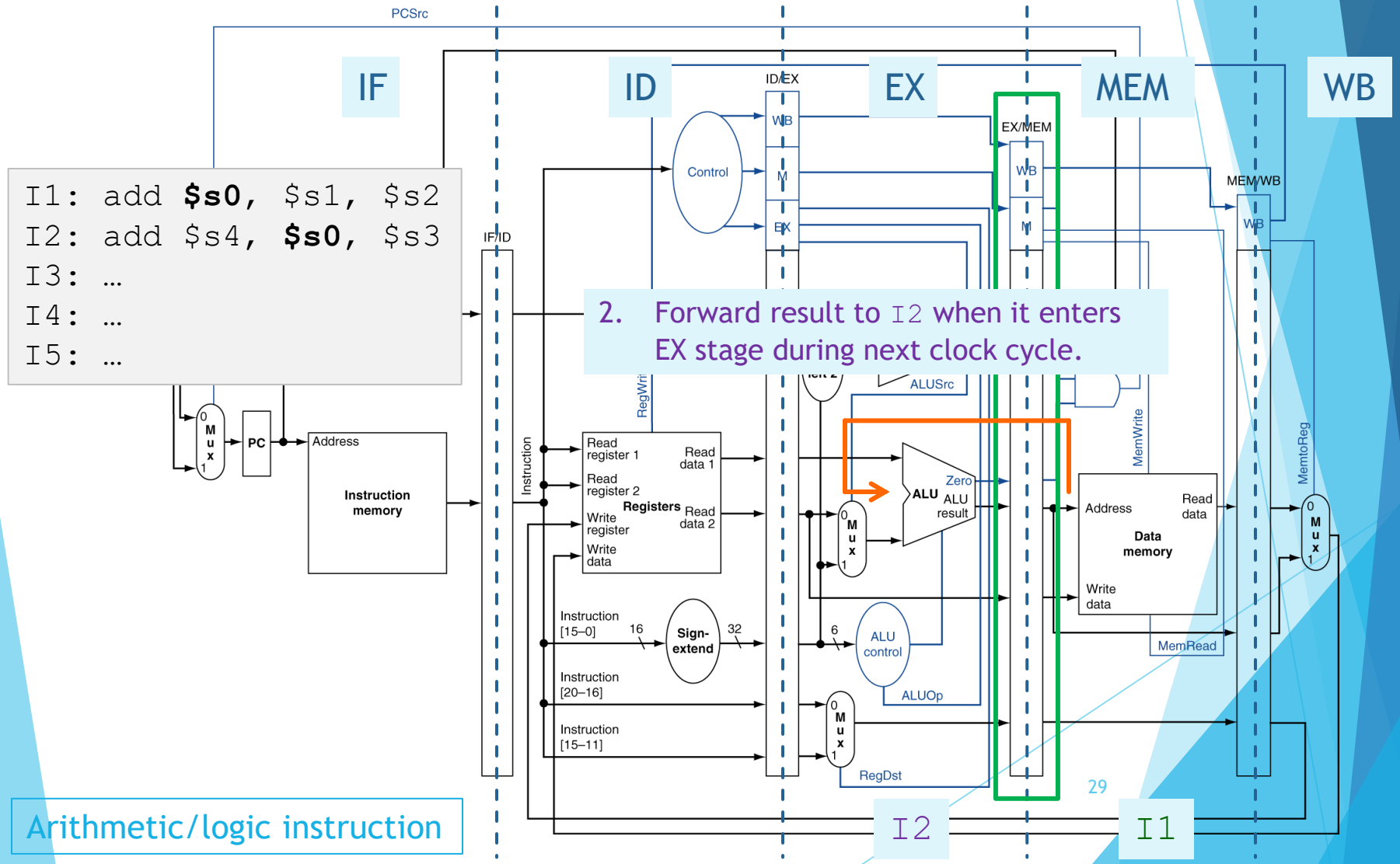


Arithmetic/logic instruction

Forwarding: Arith/Logic Instruction

```
I1: add $s0, $s1, $s2
I2: add $s4, $s0, $s3
I3: …
I4: …
I5: …
```

2. Forward result to `I2` when it enters EX stage during next clock cycle.



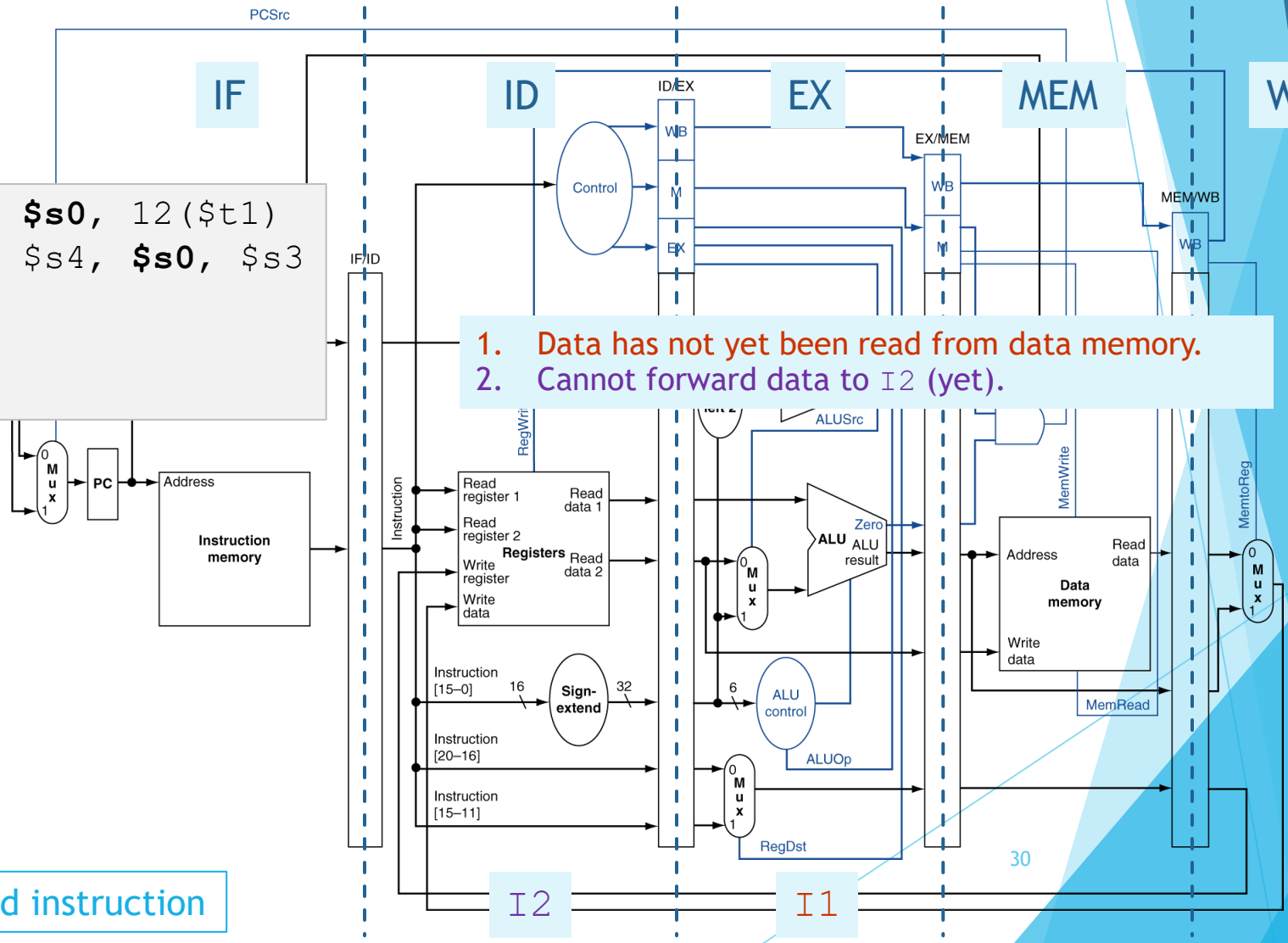
Arithmetic/logic instruction

I2 I1

Forwarding: Load Word Instruction

```
I1: lw  $s0, 12($t1)
I2: add $s4, $s0, $s3
I3: …
I4: …
I5: …
```

1. Data has not yet been read from data memory.
2. Cannot forward data to `I2` (yet).

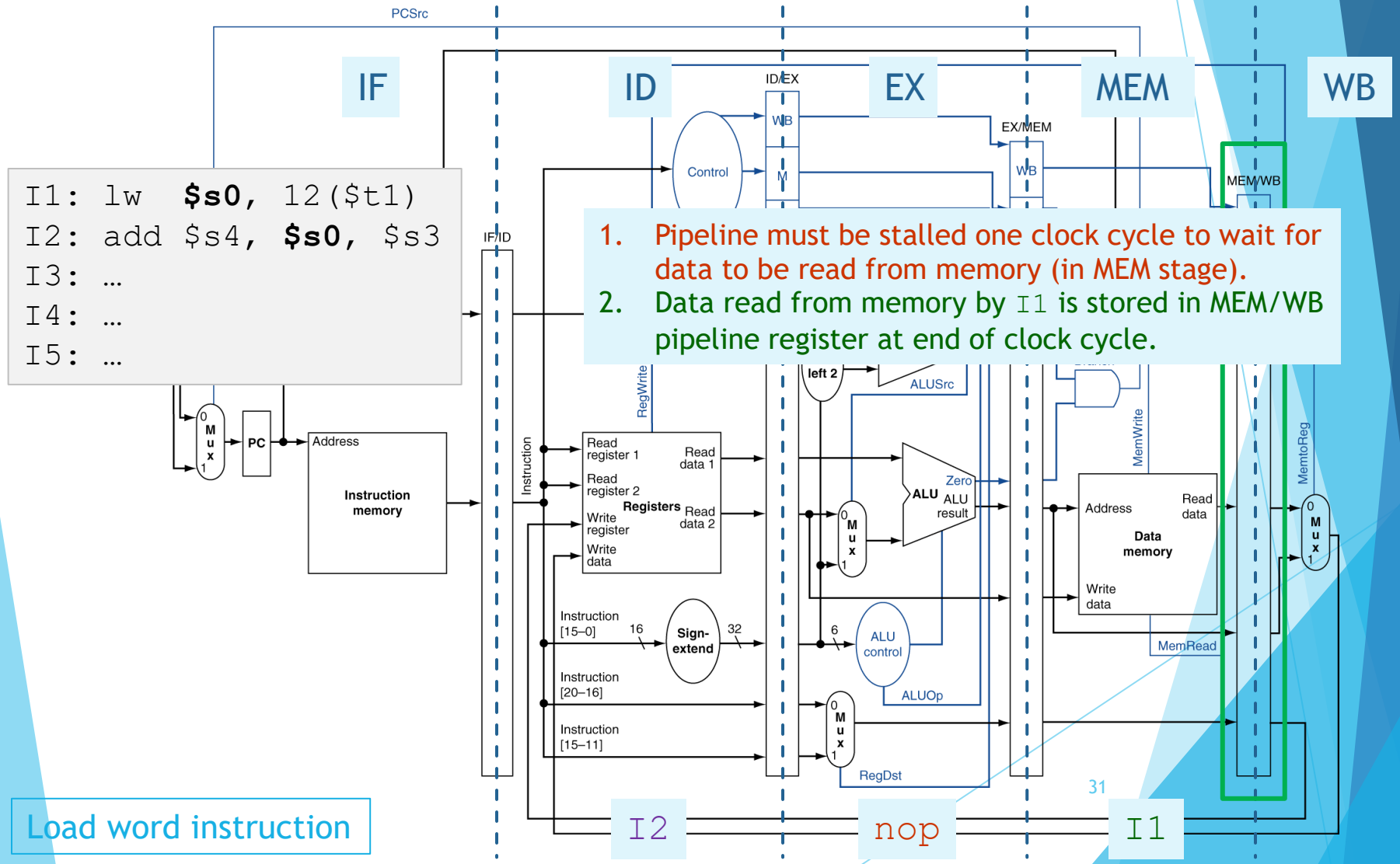


Load word instruction

Forwarding: Load Word Instruction

```
I1: lw  $s0, 12($t1)
I2: add $s4, $s0, $s3
I3: …
I4: …
I5: …
```

1. Pipeline must be stalled one clock cycle to wait for data to be read from memory (in MEM stage).
2. Data read from memory by `I1` is stored in MEM/WB pipeline register at end of clock cycle.

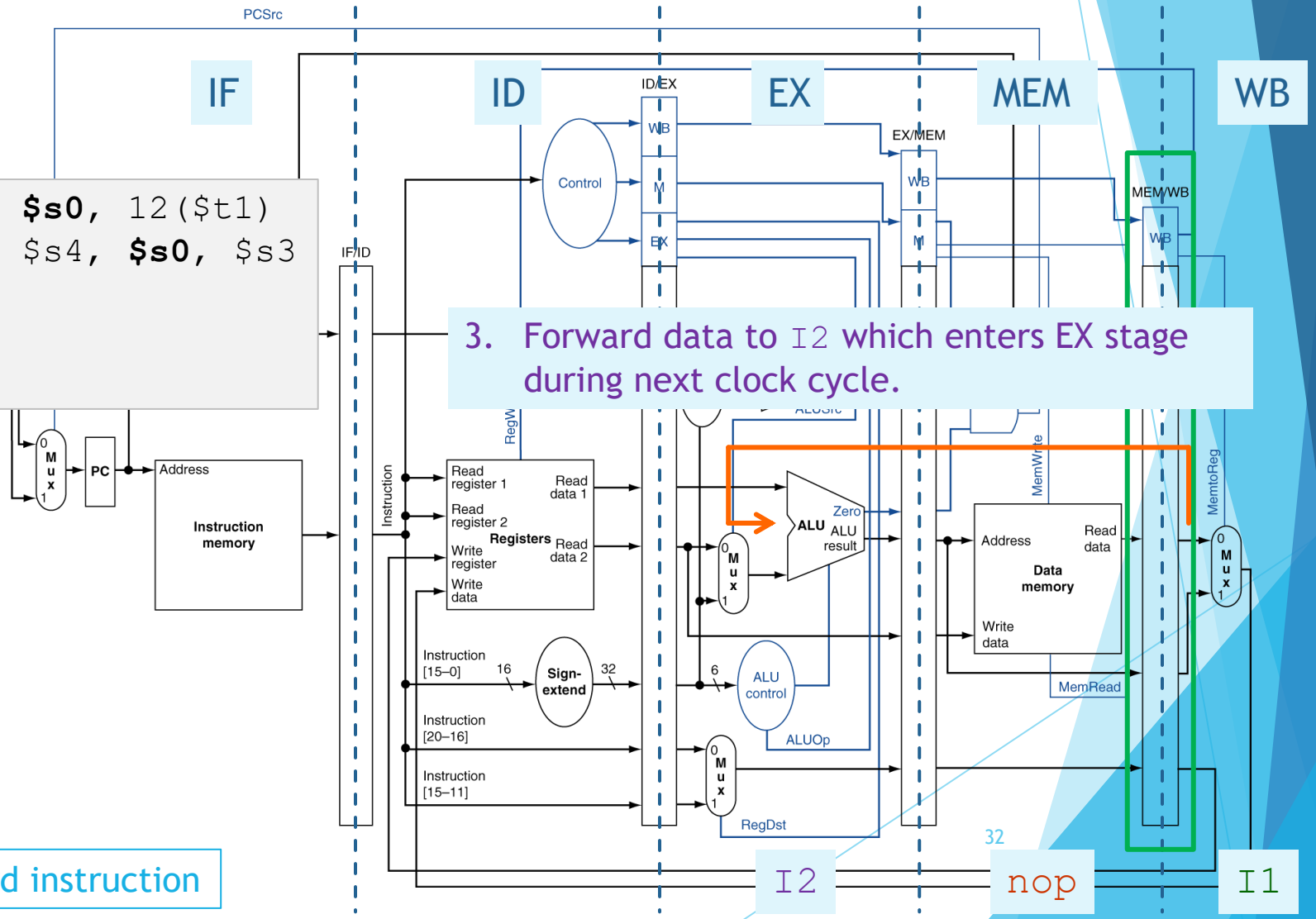


Load word instruction

Forwarding: Load Word Instruction

```
I1: lw  $s0, 12($t1)
I2: add $s4, $s0, $s3
I3: …
I4: …
I5: …
```

3. Forward data to `I2` which enters EX stage during next clock cycle.

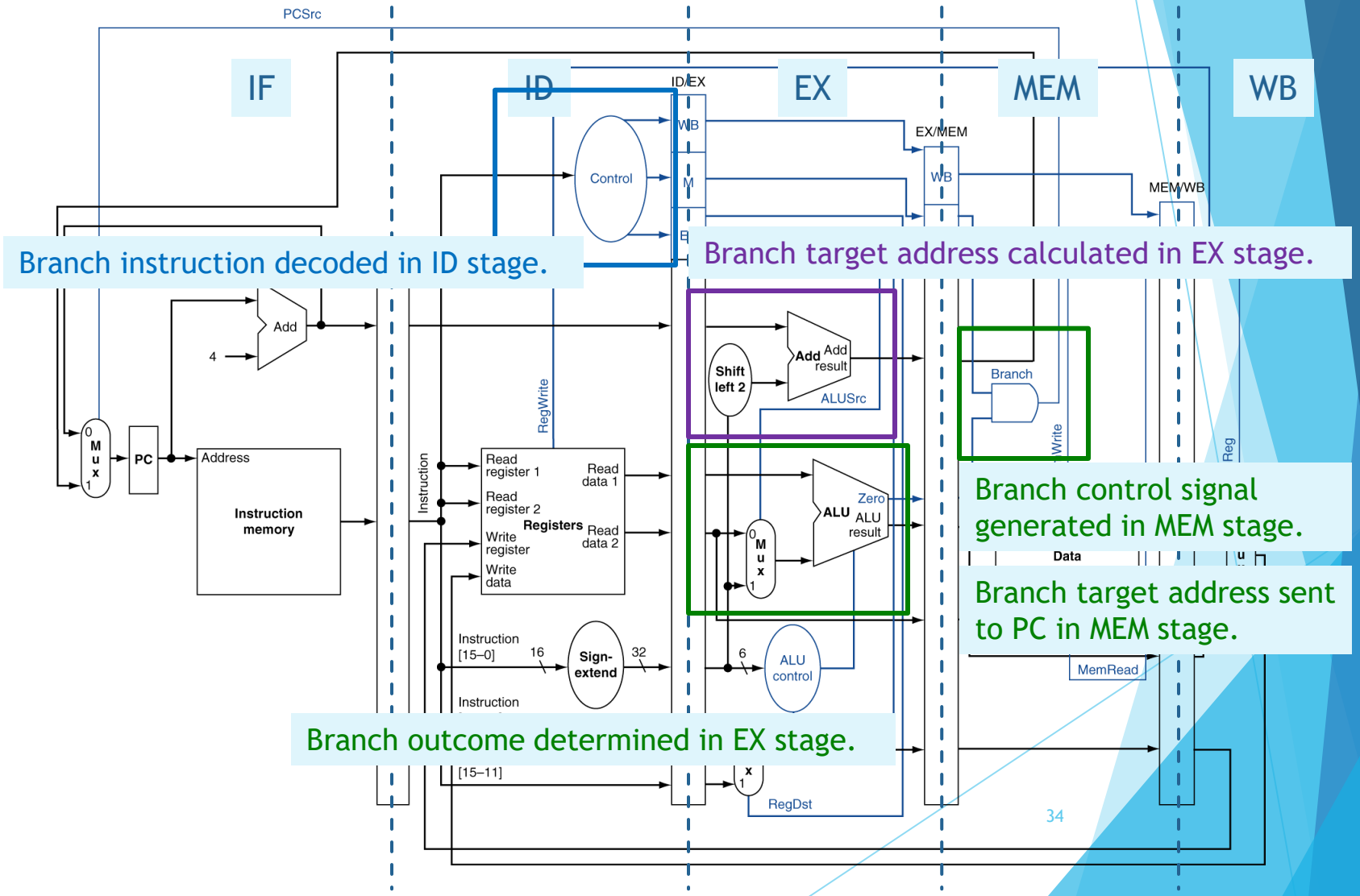


Load word instruction

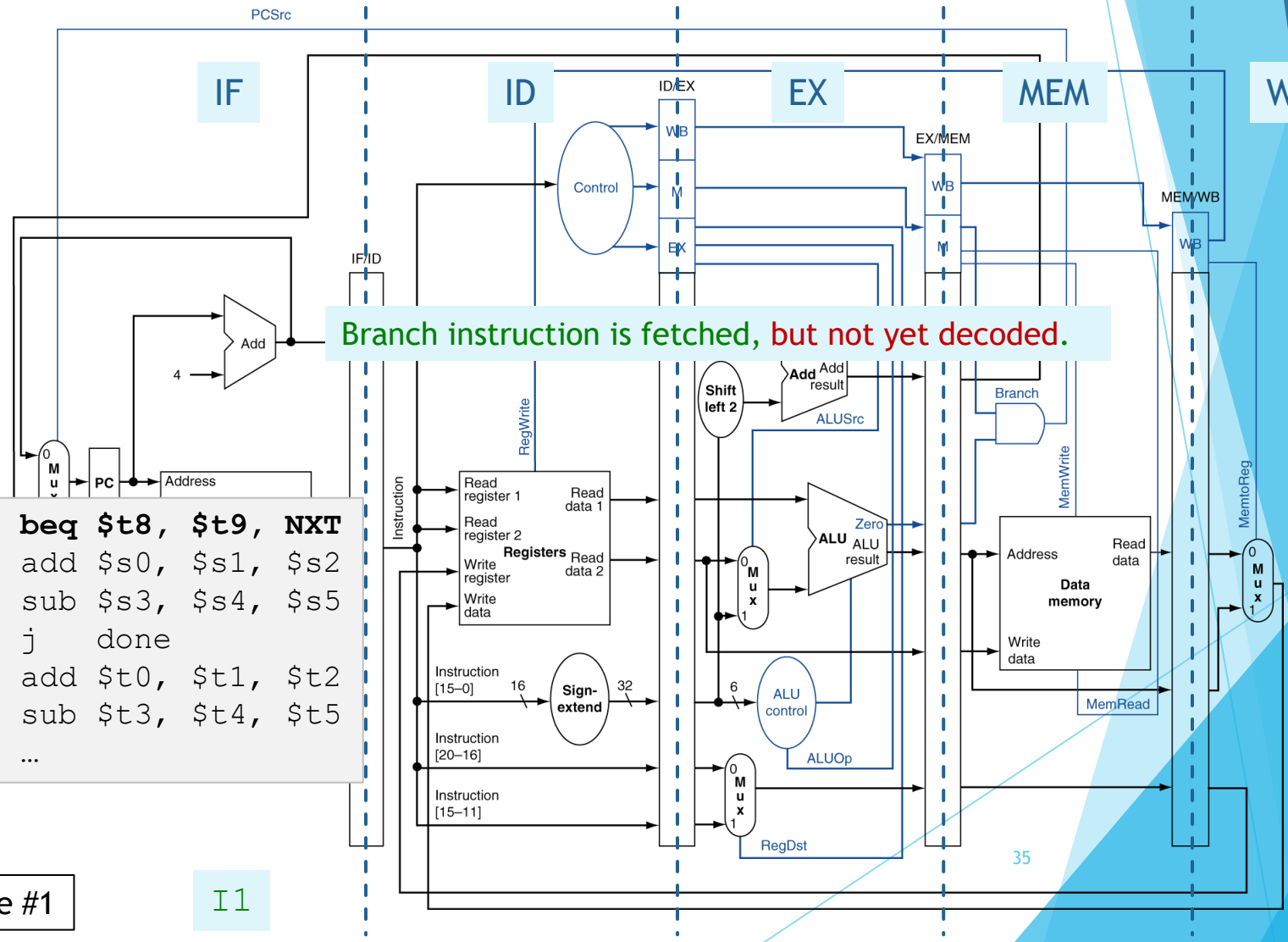
Control Hazard

- Typically associated with a **branch** instruction.
  - Branch instructions alter the flow of control (of the program).
  - Instruction to fetch next is dependent on outcome of branch instruction.
- Does not cause a problem in non-pipelined processors.
  - Branch instruction is completed before next instruction is fetched.
- However, has a negative impact on the performance of pipelined processors.
  - Multiple instructions processed concurrently in the pipeline.
  - PC must be updated before outcome of branch instruction is known.
  - May result in the wrong instruction(s) being fetched.

Control Hazard: Branch Instruction



Control Hazard: Branch Instruction



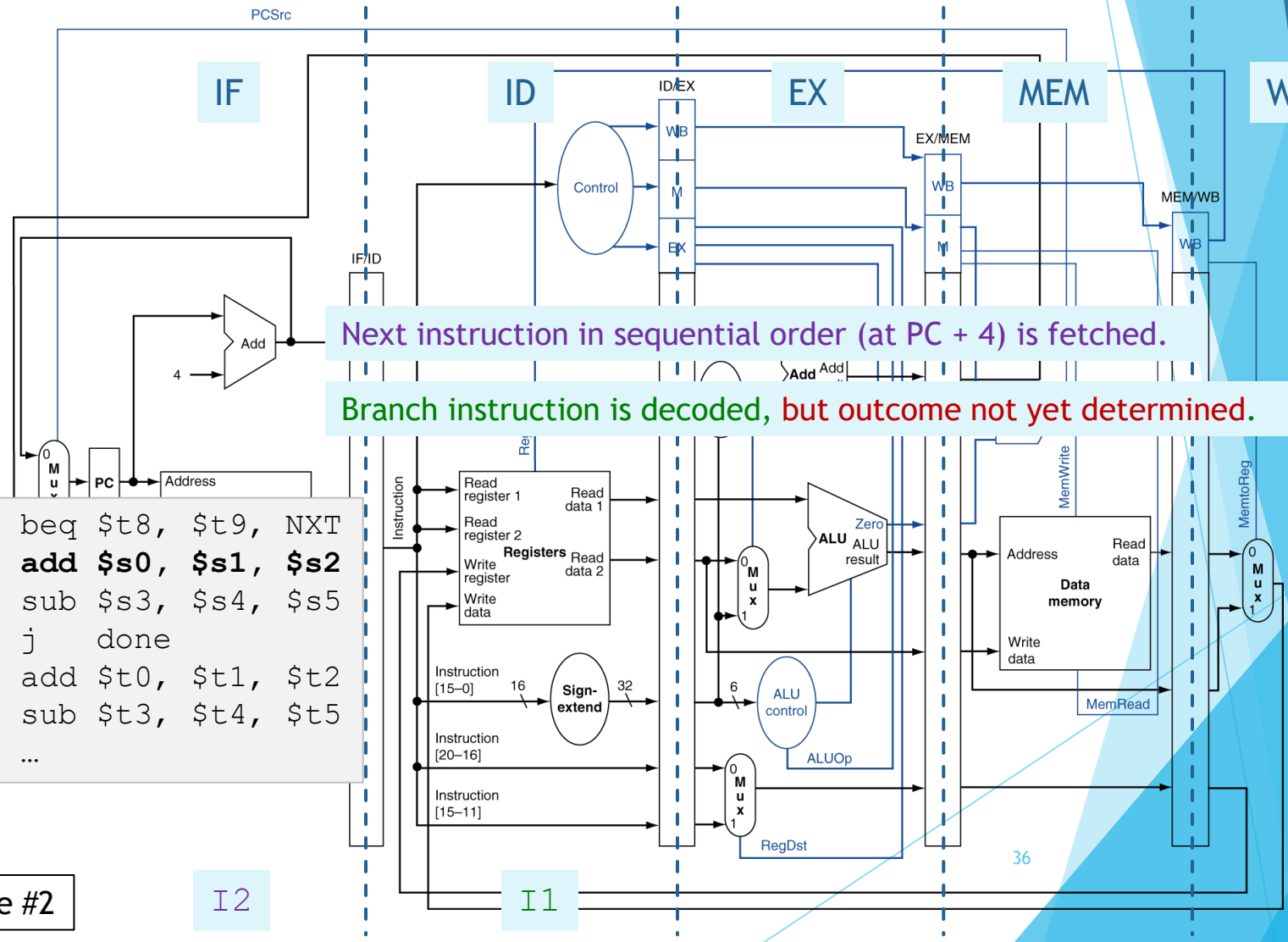
Branch instruction is fetched, but not yet decoded.

```
I1:        beq $t8, $t9, NXT
I2:        add $s0, $s1, $s2
I3:        sub $s3, $s4, $s5
I4:        j   done
I5:  NXT:  add $t0, $t1, $t2
I6:        sub $t3, $t4, $t5
I7:   DN:  …
```

Clock Cycle #1

I1

Control Hazard: Branch Instruction



Next instruction in sequential order (at PC + 4) is fetched.

Branch instruction is decoded, but outcome not yet determined.

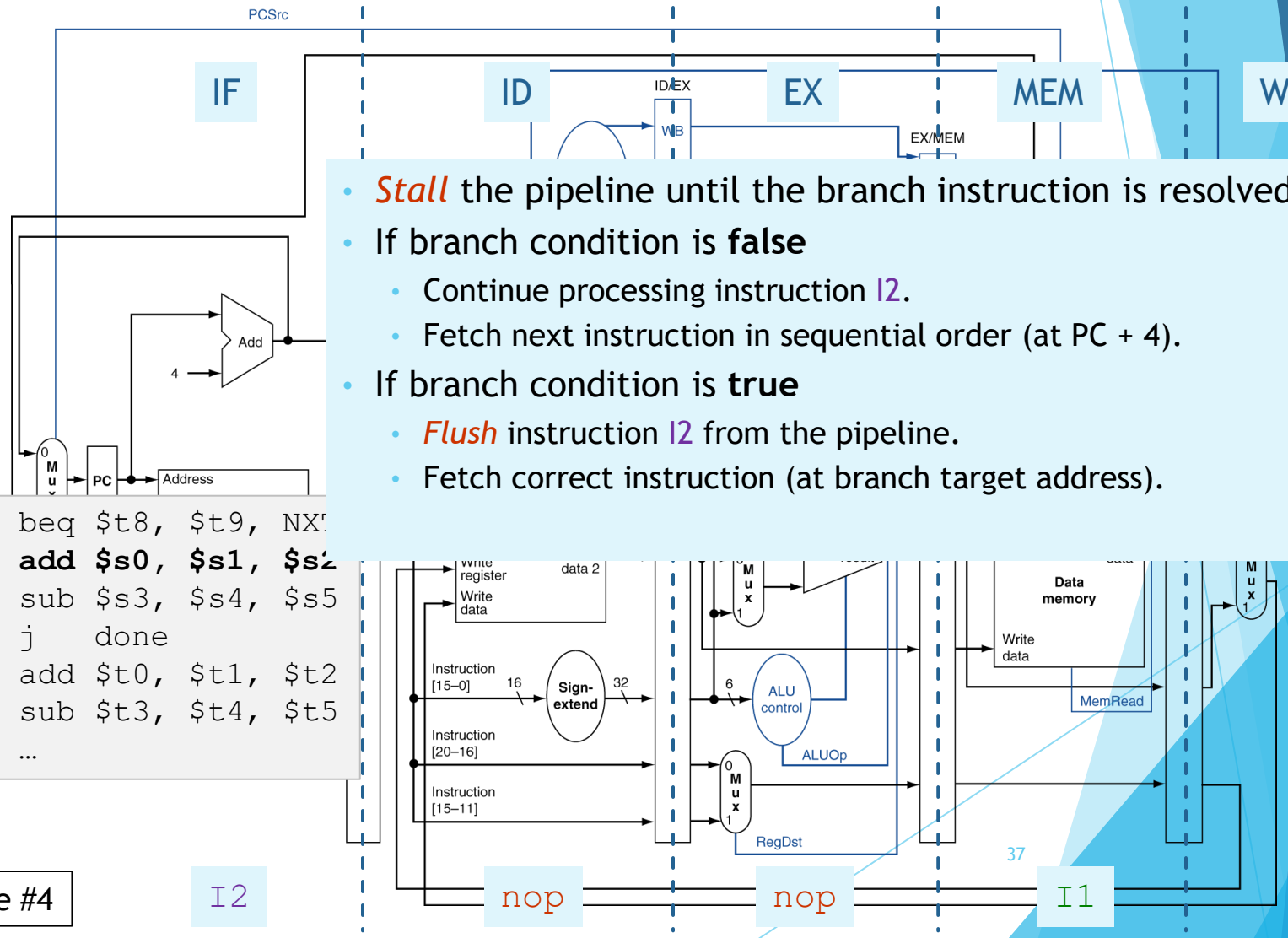
```
I1:        beq $t8, $t9, NXT
I2:        add $s0, $s1, $s2
I3:        sub $s3, $s4, $s5
I4:        j   done
I5:  NXT:  add $t0, $t1, $t2
I6:        sub $t3, $t4, $t5
I7:   DN:  …
```

Clock Cycle #2

I2

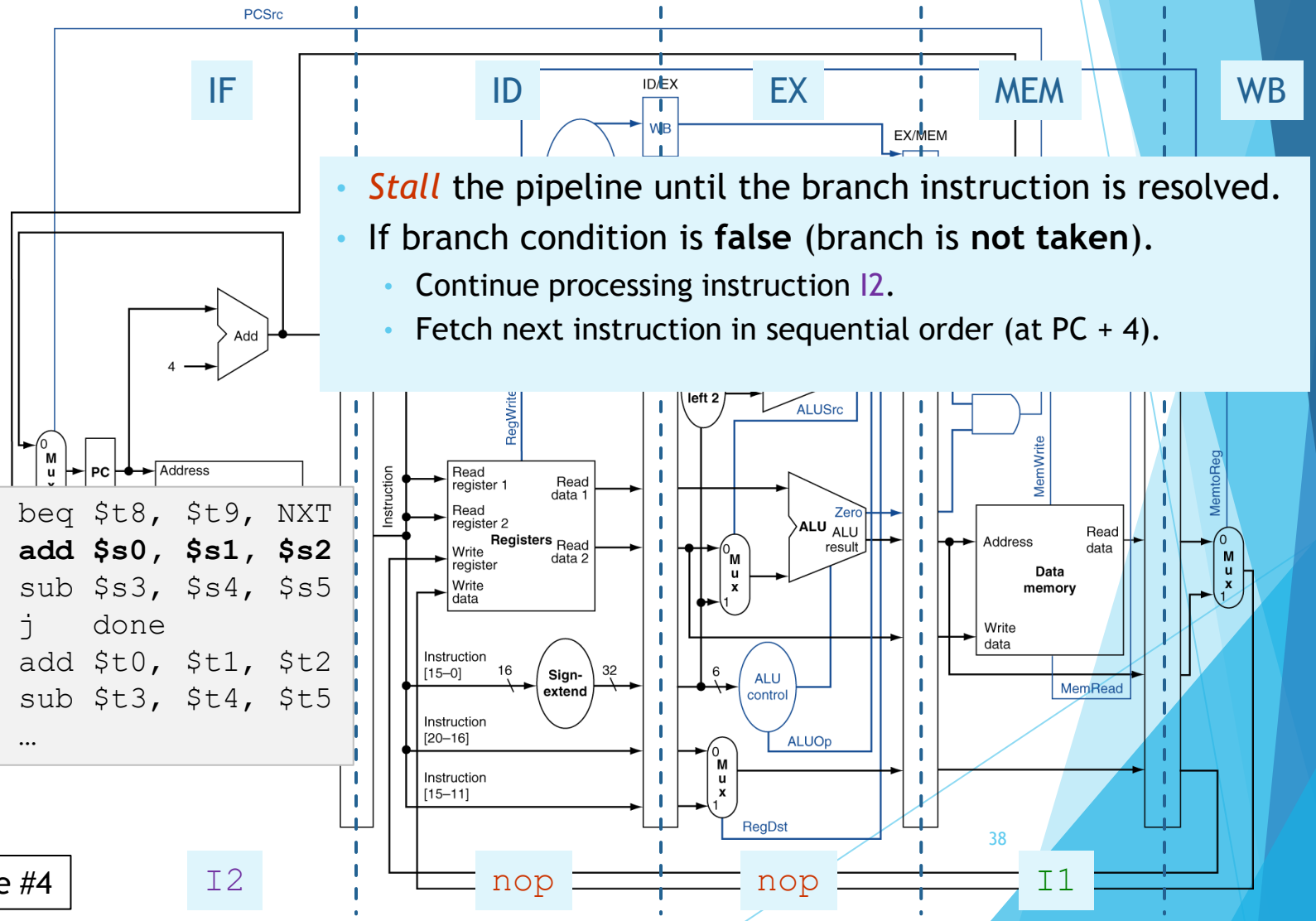
I1

Control Hazard: Branch Instruction



- *Stall* the pipeline until the branch instruction is resolved.
- If branch condition is **false**
  - Continue processing instruction I2.
  - Fetch next instruction in sequential order (at PC + 4).
- If branch condition is **true**
  - *Flush* instruction I2 from the pipeline.
  - Fetch correct instruction (at branch target address).

Control Hazard: Stall Pipeline

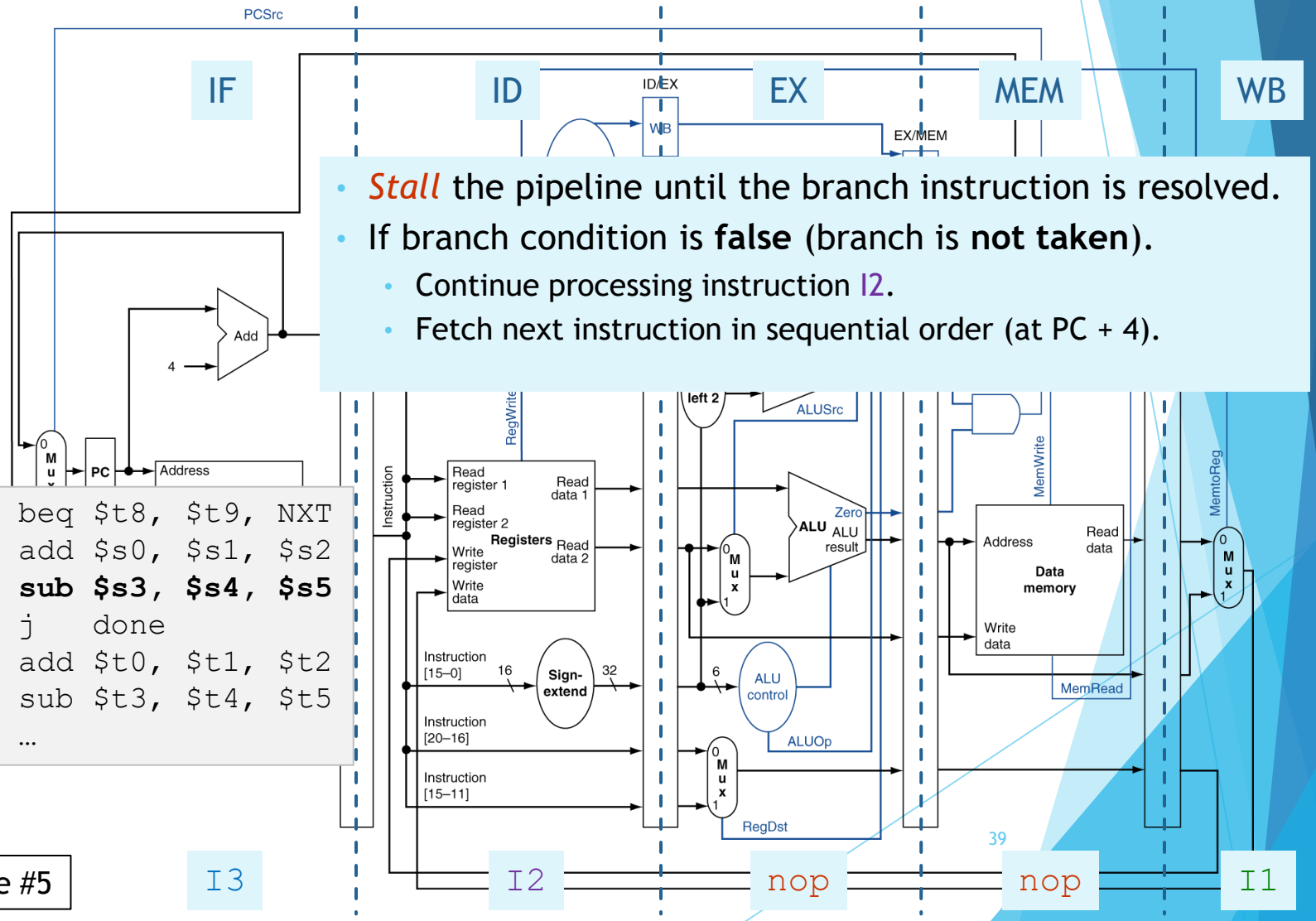


- *Stall* the pipeline until the branch instruction is resolved.
- If branch condition is **false** (branch is **not taken**).
  - Continue processing instruction I2.
  - Fetch next instruction in sequential order (at PC + 4).

```
I1:        beq $t8, $t9, NXT
I2:        add $s0, $s1, $s2
I3:        sub $s3, $s4, $s5
I4:        j   done
I5:  NXT:  add $t0, $t1, $t2
I6:        sub $t3, $t4, $t5
I7:   DN:  …
```

Clock Cycle #4

Control Hazard: Stall Pipeline

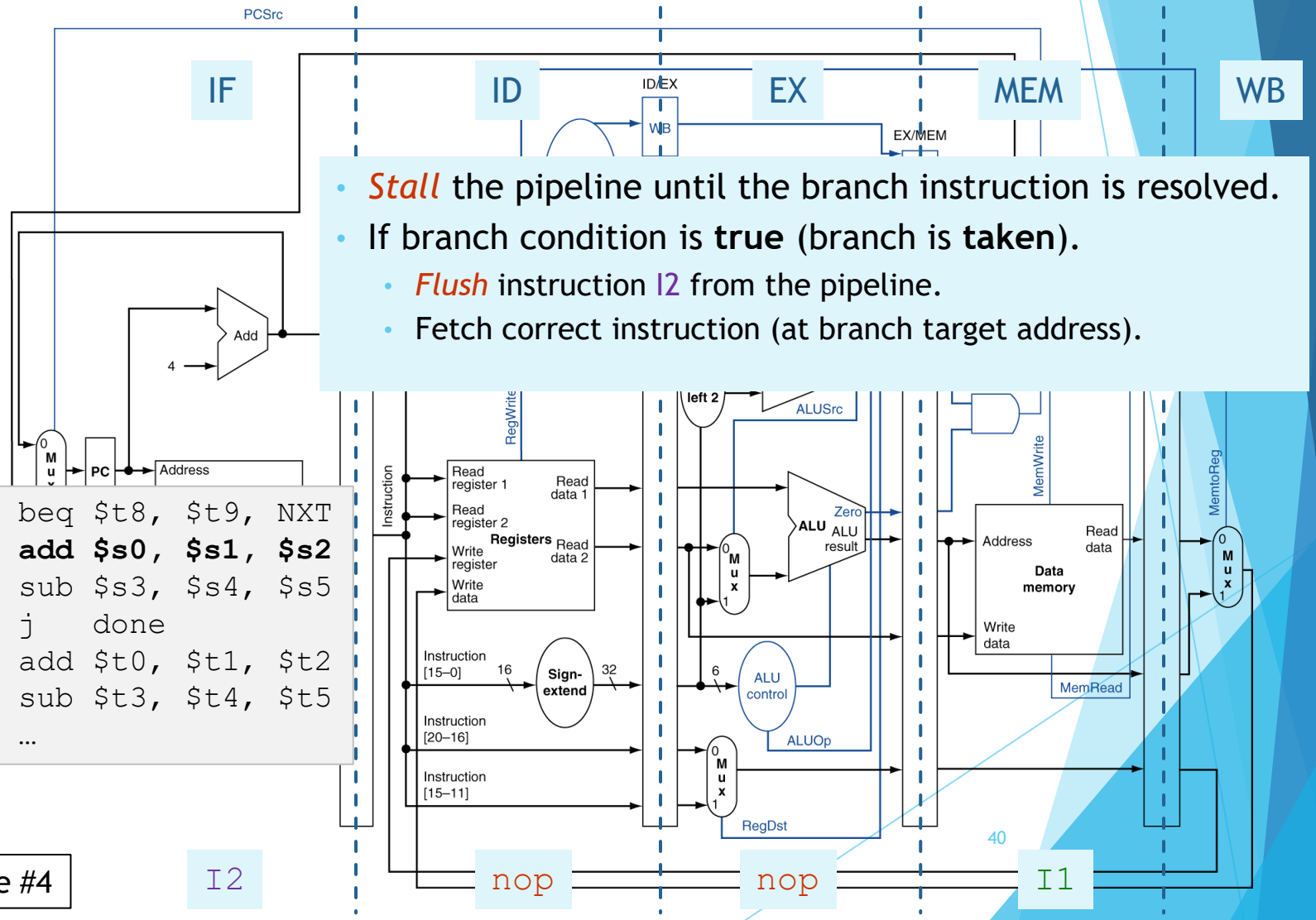


- *Stall* the pipeline until the branch instruction is resolved.
- If branch condition is **false** (branch is **not taken**).
  - Continue processing instruction I2.
  - Fetch next instruction in sequential order (at PC + 4).

```
I1:        beq $t8, $t9, NXT
I2:        add $s0, $s1, $s2
I3:        sub $s3, $s4, $s5
I4:        j   done
I5:  NXT:  add $t0, $t1, $t2
I6:        sub $t3, $t4, $t5
I7:   DN:  …
```

Clock Cycle #5

Control Hazard: Stall and Flush Pipeline



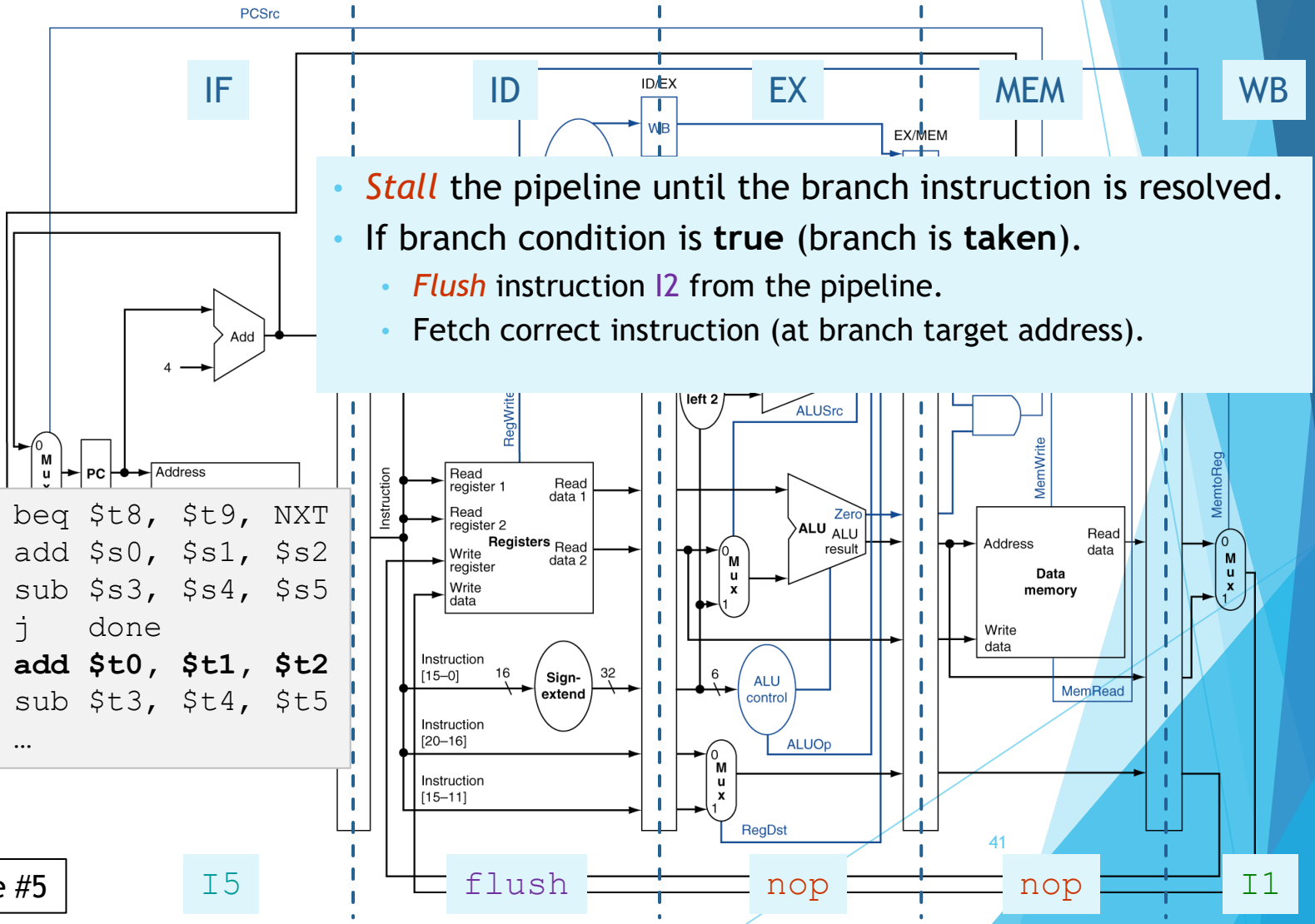
- *Stall* the pipeline until the branch instruction is resolved.
- If branch condition is **true** (branch is **taken**).
  - *Flush* instruction I2 from the pipeline.
  - Fetch correct instruction (at branch target address).

```
I1:        beq $t8, $t9, NXT
I2:        add $s0, $s1, $s2
I3:        sub $s3, $s4, $s5
I4:        j   done
I5:  NXT:  add $t0, $t1, $t2
I6:        sub $t3, $t4, $t5
I7:   DN:  …
```

Clock Cycle #4

Control Hazard: Stall and Flush Pipeline

- *Stall* the pipeline until the branch instruction is resolved.
- If branch condition is **true** (branch is **taken**).
  - *Flush* instruction I2 from the pipeline.
  - Fetch correct instruction (at branch target address).




```
I1:        beq $t8, $t9, NXT
I2:        add $s0, $s1, $s2
I3:        sub $s3, $s4, $s5
I4:        j   done
I5:  NXT:  add $t0, $t1, $t2
I6:        sub $t3, $t4, $t5
I7:   DN:  …
```

Clock Cycle #5

I5 | flush | nop | nop | I1

Control Hazard: Stall Pipeline

- Consider the *branch* instruction in the code given below.
- Stall pipeline when appropriate.
- Flush instructions from pipeline when needed.

```
        beq $t8, $t9, next   (I1)
        add $s0, $s1, $s2    (I2)
        sub $s3, $s4, $s5    (I3)
        j   done             (I4)
next:   add $t0, $t1, $t2    (I5)
        sub $t3, $t4, $t5    (I6)
done:   …                    (I7)
```

Assume `$t8 != $t9`.

time

	IF	ID	EX	MEM	WB
1					
2					
3					
4					
5					
6					
7					
8					
9					
10					

Control Hazard: Stall Pipeline

- Consider the *branch* instruction in the code given below.
- Stall pipeline when appropriate.
- Flush instructions from pipeline when needed.

```
        beq $t8, $t9, next   (I1)
        add $s0, $s1, $s2    (I2)
        sub $s3, $s4, $s5    (I3)
        j   done             (I4)
next:   add $t0, $t1, $t2    (I5)
        sub $t3, $t4, $t5    (I6)
done:   …                    (I7)
```

Branch instruction fetched from memory.

Clock Cycle #1

time

	IF	ID	EX	MEM	WB
1	I1				
2					
3					
4					
5					
6					
7					
8					
9					
10					

Control Hazard: Stall Pipeline

- Consider the *branch* instruction in the code given below.
- Stall pipeline when appropriate.
- Flush instructions from pipeline when needed.

```
        beq $t8, $t9, next   (I1)
        add $s0, $s1, $s2    (I2)
        sub $s3, $s4, $s5    (I3)
        j   done             (I4)
next:   add $t0, $t1, $t2    (I5)
        sub $t3, $t4, $t5    (I6)
done:   …                    (I7)
```

time

Branch instruction decoded.
Next instruction in sequential order fetched.

Clock Cycle #2

	IF	ID	EX	MEM	WB
1	I1				
2	I2	I1			
3					
4					
5					
6					
7					
8					
9					
10					

Control Hazard: Stall Pipeline

- Consider the *branch* instruction in the code given below.
- Stall pipeline when appropriate.
- Flush instructions from pipeline when needed.

```
        beq $t8, $t9, next   (I1)
        add $s0, $s1, $s2    (I2)
        sub $s3, $s4, $s5    (I3)
        j   done             (I4)
next:   add $t0, $t1, $t2    (I5)
        sub $t3, $t4, $t5    (I6)
done:   …                    (I7)
```

Stall pipeline until branch is resolved.
(Branch is resolved in MEM stage.)

Clock Cycle #3 - #4

time

	IF	ID	EX	MEM	WB
1	I1				
2	I2	I1			
3	I2	NOP	I1		
4	I2	NOP	NOP	I1	
5					
6					
7					
8					
9					
10					

Control Hazard: Stall Pipeline

- Consider the *branch* instruction in the code given below.
- Stall pipeline when appropriate.
- Flush instructions from pipeline when needed.

```
        beq $t8, $t9, next   (I1)
        add $s0, $s1, $s2    (I2)
        sub $s3, $s4, $s5    (I3)
        j   done             (I4)
next:   add $t0, $t1, $t2    (I5)
        sub $t3, $t4, $t5    (I6)
done:   …                    (I7)
```

Since branch is **not taken**, continue processing instruction `I2` and fetch instruction `I3`.

Clock Cycle #5

time

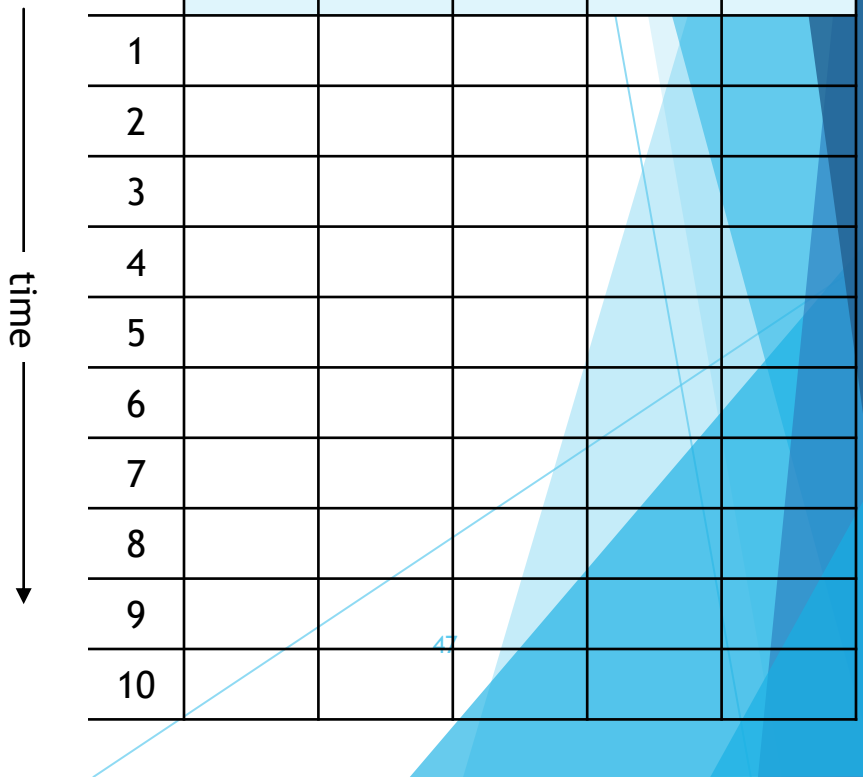
	IF	ID	EX	MEM	WB
1	I1				
2	I2	I1			
3	I2	NOP	I1		
4	I2	NOP	NOP	I1	
5	I3	I2	NOP	NOP	I1
6					
7					
8					
9					
10					

Control Hazard: Stall Pipeline

- Consider the *branch* instruction in the code given below.
- Stall pipeline when appropriate.
- Flush instructions from pipeline when needed.

```
        beq $t8, $t9, next   (I1)
        add $s0, $s1, $s2    (I2)
        sub $s3, $s4, $s5    (I3)
        j   done             (I4)
next:   add $t0, $t1, $t2    (I5)
        sub $t3, $t4, $t5    (I6)
done:   …                    (I7)
```

Assume `$t8 == $t9`.



Control Hazard: Stall Pipeline

- Consider the *branch* instruction in the code given below.
- Stall pipeline when appropriate.
- Flush instructions from pipeline when needed.

```
        beq $t8, $t9, next   (I1)
        add $s0, $s1, $s2    (I2)
        sub $s3, $s4, $s5    (I3)
        j   done             (I4)
next:   add $t0, $t1, $t2    (I5)
        sub $t3, $t4, $t5    (I6)
done:   …                    (I7)
```

Branch instruction decoded.

	IF	ID	EX	MEM	WB
1	I1				
2	I2	I1			
3	I2	NOP	I1		
4	**FSH**	NOP	NOP	I1	
5	I5	**FSH**	NOP	NOP	I1
6	I6	I5	**FSH**	NOP	NOP
7	I7	I6	I5	**FSH**	NOP
8		I7	I6	I5	**FSH**
9			I7	I6	I5
10				I7	I6

time

Branch instruction resolved.

Incorrect instruction fetched.

Incorrect instruction flushed from pipeline.

Fetch instruction at branch target address.